

UNIT 3, AGENT DEVELOPMENT WITH PYTHON AND FRAMEWORKS

LECTURE 2

Semantic Kernel Foundations

The foundation layer that survived the merger. Kernel, plugins, connectors. You built every one of these by hand in Unit 1, and now you meet the industrial version.

SECTION 00

Not dead knowledge

Semantic Kernel became the base of Agent Framework. Learn it properly.

RECALL

Why we learn a maintenance-mode framework

THE WORRY

Is this a waste of time?

Last lecture I said Semantic Kernel is in maintenance mode. So why spend a whole lecture on it? Is this dead knowledge?

THE ANSWER

It is the foundation of the live thing

Semantic Kernel survived the merger as the base layer of Microsoft Agent Framework. Its kernel, plugins and connectors sit underneath the framework you will build on. Learn it, and you understand the ground the whole tower stands on.

THE HONEST FRAMING

You are not learning Semantic Kernel to ship a Semantic Kernel app. You are learning the three ideas, kernel, plugin, connector, that the modern framework inherited unchanged. These concepts are current even though the standalone package is not. That distinction is worth holding in your head.

Today

- The kernel: the central object that holds your tools and your model connection.
- Plugins: the real, current way to give an agent tools, and how they beat hand-written schemas.
- The surprise that makes this easy: plugin functions run without a model.
- Connectors: how the kernel reaches a model, which is just your lanes again.
- Every piece mapped back to the code you wrote in Unit 1.

HOLD ON TO THIS

The running example is the same weather and triage domain, rebuilt as real Semantic Kernel plugins. Same behaviour you already know, now expressed in the framework the industry actually uses.

SECTION 01

The kernel and the plugin

Two objects, and you built both before.

The kernel is your dispatch loop, promoted

WHAT YOU BUILT

A dispatch loop

In Unit 1 you had a REGISTRY dict and a loop that looked up a tool name and called the matching function. You wrote the lookup, the call, the whole thing.

WHAT THE KERNEL IS

That loop, as an object

The Kernel holds your functions and owns the lookup and dispatch. You register plugins onto it, and it handles finding and calling the right function. Same job, given a home and a name.

```
kernel = Kernel()
kernel.add_plugin(WeatherPlugin(), plugin_name="weather")
kernel.add_plugin(TicketPlugin(), plugin_name="tickets")
```

A plugin is a class with decorated methods

```
class WeatherPlugin:

    @kernel_function(
        name="get_weather",
        description="Get current weather for one city.")
    def get_weather(
        self,
        city: Annotated[str, "The city name."]) -> str:
        return self._DATA.get(city.lower(), "no data")
```

YOU HAVE SEEN EVERY PART OF THIS

The method body is an ordinary function, exactly like your Unit 1 tools. The `@kernel_function` decorator plus the `Annotated` type is your `TOOL_SCHEMA`, the name, description, and typed parameter you used to write by hand in a dict. The only change: the framework reads it off your annotations instead of you maintaining a separate schema.

Why this beats a hand-written schema

UNIT 1, BY HAND

Two things that drift

You wrote the function, and separately you wrote a `TOOL_SCHEMA` dict describing it. Change one and forget the other, and the model is now lied to. Your Lecture 5 linter existed partly to catch this.

SEMANTIC KERNEL

One source of truth

The schema is generated from your type hints and docstring. There is nothing separate to keep in sync, because there is only one description of the function, on the function. Drift becomes impossible.

HOLD ON TO THIS

Remember the linter from Unit 1 Lecture 5 that checked your schema matched your function? The plugin model removes the need for it entirely, because the schema and the function can no longer disagree. The framework solved a whole class of your bugs by construction.

SECTION 02

The surprise that makes this easy

You do not need a model to run a plugin.

The model only chooses. It does not call.

A plugin function is ordinary Python. The kernel can run it with no model at all.

```
# no model, no token, no lane. just call the function.  
result = await kernel.invoke(  
    kernel.get_function("weather", "get_weather"),  
    KernelArguments(city="Mumbai"))  
# -> "Mumbai: 31C, humid, light rain."
```

WHY THIS MATTERS SO MUCH

The model's only job is to choose which function to call. The calling is ordinary code the kernel dispatches. So the entire plugin layer, the part you actually write, is testable without spending a token. Your whole tool suite can be unit tested offline, and only the routing decision needs a live model. This is the single most useful fact in the lecture.

The schema, generated not written

```
>>> function_metadata(kernel, "weather", "get_weather")
{
  "name": "get_weather",
  "description": "Get the current weather for one city",
  "parameters": [
    {"name": "city", "description": "The city name."}
  ]
}
```

COMPARE THIS TO WHAT YOU WROTE IN UNIT 1

This is the exact TOOL_SCHEMA dict you used to write by hand, name, description, typed parameters. But you did not write it. Your decorator and your type annotations produced it. One source of truth, generated on demand, never out of sync with the function it describes. That is the plugin model earning its keep.

SECTION 03

Connectors

How the kernel reaches a model. It is your lanes, again.

A connector is a lane with a framework name

YOUR LANES

get_client()

One function that returns a client pointed at a model, configured by PROVIDER and a key. You built it to make the code provider-proof across four lanes.

SK CONNECTORS

OpenAIChatCompletion(...)

The kernel's way to reach a model. Same idea: a model id, a key, a base url. You add it to the kernel, and now the kernel can ask the model to choose a function.

```
service = OpenAIChatCompletion(  
    ai_model_id=MODEL, async_client=your_lane_client)
```

THE FULL PICTURE

Kernel holds plugins (your tools) and a connector (your lane). Ask it to handle a request, and it shows the model your plugin schemas, the model picks one, and the kernel calls it. That is your entire Unit 1 agent loop, owned by one object you configured in three lines.

Hold on to this

Kernel, plugin, connector. You built all three by hand. Now you know their real names.

The kernel is your dispatch loop

It holds plugins and a connector, and owns the lookup.

A plugin is a decorated class

The decorator plus annotations is your Unit 1 schema, generated.

Plugins run without a model

So your whole tool layer is testable offline, for free.

A connector is a lane

Model id, key, base url. The kernel reaches a model through it.

Before next session

DO

Write a real plugin

Build a plugin with two kernel functions of your own, register it, and invoke both directly with no model. Confirm your whole tool layer runs and is testable offline.

DO

Read the generated schema

Print `function_metadata` for your plugin. Compare it, field by field, to a `TOOL_SCHEMA` dict you wrote by hand in Unit 1. Notice you wrote none of it this time.

THINK

Where did the linter go

Your Unit 1 linter checked the schema matched the function. Explain in two sentences why the plugin model makes that linter unnecessary. This is a real understanding check.

ON THE THIRD ONE

If you can explain why the plugin model retires your linter, you understand what frameworks really buy you. They do not just save typing. The good ones make whole categories of bug impossible by construction. That is the difference between a convenience and a foundation.

Next session

Unit 3, Lecture 3

Building on Microsoft Agent Framework

You have the foundation. Next we build a real agent on the successor framework itself, the one you would ship. Plugins become tools it can call, the connector becomes its brain, and async execution enters properly. Everything you learned today sits underneath it, which is exactly why we learned the foundation first.

COME WITH

Your own two-function plugin, its generated schema printed, and your explanation of why the linter is gone.